VIETTEL DIGITAL TALENT 2026

Streaming Processing in BigData Platform

Real-time data pipelines with Apache Kafka & Apache Flink

Mentor: Nguyen Minh Hieu
Student: Ngo Quang Hieu

Data Engineer Track
Viettel Digital Talent 2026

July 1^{st}, 2026

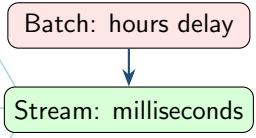
Table of Contents

Agenda

Why Stream Processing?

The Problem

- **Batch pipelines** — hours/days latency
- **Time-sensitive data** — stale = lost value
- **High velocity** — thousands of events/sec
- **No replay** — failures cause data loss



Stream Processing Enables

- **Real-time personalisation** — react in ms
- **Live dashboards** — current state, not snapshot
- **Fault tolerance** — exactly-once guarantees
- **Decoupled services** — event-driven architecture

This Project: simulate a music streaming platform generating **3 event types** and process them end-to-end in real time.

viettel
Theo cách của bạn

Objectives & Scope

Objectives

1. Understand **stream-processing** concepts
2. Evaluate **Apache Kafka** as event transport
3. Compare **Spark vs Flink**
4. Build **Streamify** — full ELT pipeline

In Scope

- Kafka → Flink → Snowflake pipeline
- Kafka: ZooKeeper → KRaft evolution
- Spark micro-batch vs Flink true-streaming
- dbt star schema + Airflow scheduling
- Local Docker Compose deployment

Out of Scope

- Real-time ML / recommendations
- Cloud / Kubernetes deployment
- Schema evolution (Avro/CDC)

Agenda

Why Apache Kafka?

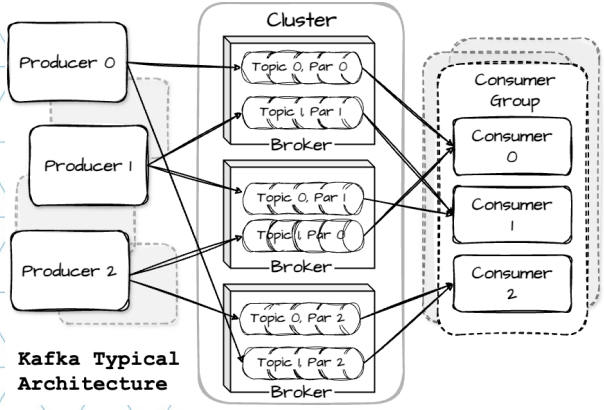
Problems Without a Message Queue

- **Tight coupling** — producer knows every consumer
- **No buffering** — slow consumer $\rightarrow$ data loss
- **No replay** — consumed $=$ gone forever
- **Fan-out cost** — N consumers $=$ N calls

How Kafka Solves It

- **Decoupling** — write to topic, subscribe independently
- **Durable buffer** — persisted to disk, replicated
- **Replay** — configurable retention; re-read anytime
- **Scalability** — partitions scale throughput linearly
- **Fan-out free** — multiple groups, own offsets

Kafka Architecture



Kafka Typical Architecture

- **Producer** — partitions by key; batches & compresses; `acks=all` for durability
- **Broker** — leader/follower replicas; append-only log; sequential I/O
- **Topic** — immutable records; time-based or log-compaction retention
- **Partition** — ordered, offset-indexed; unit of parallelism
- **Consumer Group** — one partition → one member; independent offset per group

Max parallelism = number of partitions

Agenda

viettel
Theo cách của bạn

Batch vs. Stream Processing

Batch Processing

- Collect → store → process in bulk
- Latency: **hours to days**
- Tools: Hadoop, scheduled SQL

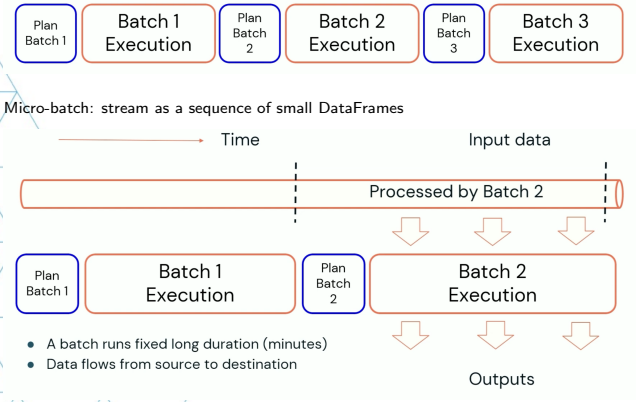
Stream Processing

- Compute *continuously* as records arrive
- Latency: **milliseconds to seconds**
- Tools: Apache Flink, Spark Streaming

Key Challenges in Streaming

1. **Unbounded data** — stream never ends; state must be bounded
2. **Out-of-order events** — need watermarks to handle late arrivals
3. **Fault tolerance** — recover without loss or duplicates; exactly-once semantics

Apache Spark Structured Streaming



Micro-batch (default)

- Trigger interval (e.g. 1 s)
- Latency: **100 ms – seconds**
- Exactly-once with idempotent sinks
- Full stateful operators

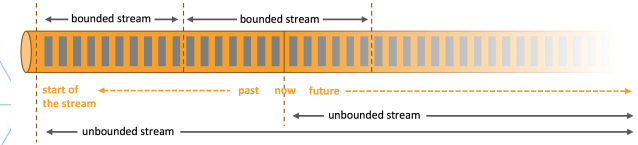
Real-time Mode (Spark 4)

- Concurrent stages + streaming shuffle
- Latency: **sub-100 ms**
- Exactly-once maintained
- Currently in public preview

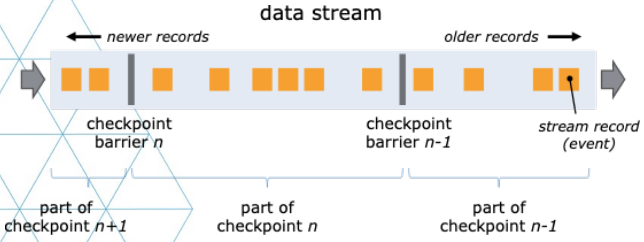
Unified API — same code for batch & streaming

viettel
Theo cách của bạn

Apache Flink: True Event Streaming



True streaming: records flow through operator DAG one-by-one



Chandy-Lamport barriers: async snapshots without pausing processing

Dataflow Model

- **Record-at-a-time** — no batching delay
- Operators form a DAG; data flows continuously
- **Event time + watermarks** — correct out-of-order handling

Fault Tolerance

- Barrier checkpoints; state in RocksDB
- Recover from last complete checkpoint
- **Exactly-once** with transactional sinks

APIs: DataStream, Table API, Flink SQL, **PyFlink**

viettel
Theo cách của bạn

Spark vs. Flink: Side-by-Side

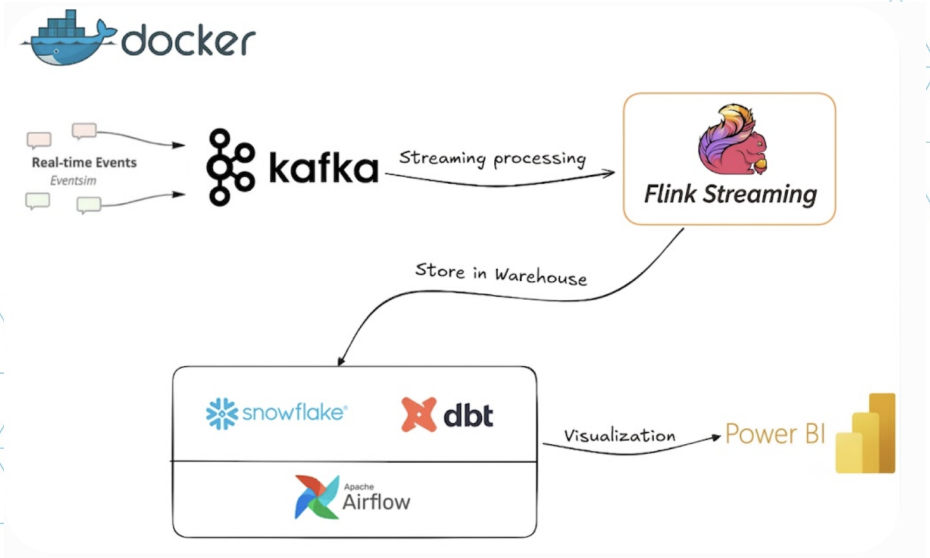
Dimension	Apache Spark	Apache Flink
Execution model	Micro-batch; Real-time Mode (Spark 4, preview)	True event-by-event streaming
Latency	100 ms–s (micro-batch); sub-100 ms (RTM)	1–10 ms
Fault tolerance	WAL + idempotent sinks	Async barrier snapshotting
State backend	In-memory / RocksDB (Spark 3.2+)	RocksDB (incremental) / heap
Exactly-once	Yes (micro-batch + idempotent sink)	Yes (transactional sink)
Use case	Unified batch + streaming; ML pipelines	Low-latency, stateful, CEP

Why Flink for Streamify: lower native latency, richer event-time semantics, and

`StatementSet` runs 3 Kafka→Snowflake pipelines as **one job** with one shared checkpoint.

Agenda

Streamify: End-to-End Architecture



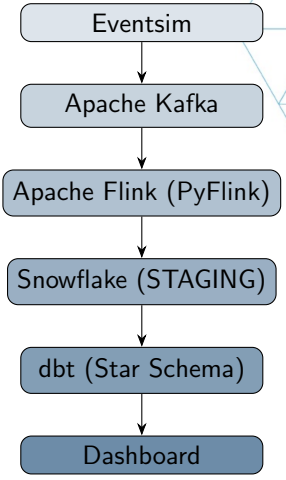
Pipeline: Ingest → Process → Model → Serve

Real-time Ingestion

- **Eventsim** — generates 3 Kafka topics: `listen_events`, `page_view_events`, `auth_events`
- **Kafka** — durable, partitioned transport
- **PyFlink** — parses JSON, enriches timestamps, writes to Snowflake via JDBC (exactly-once)

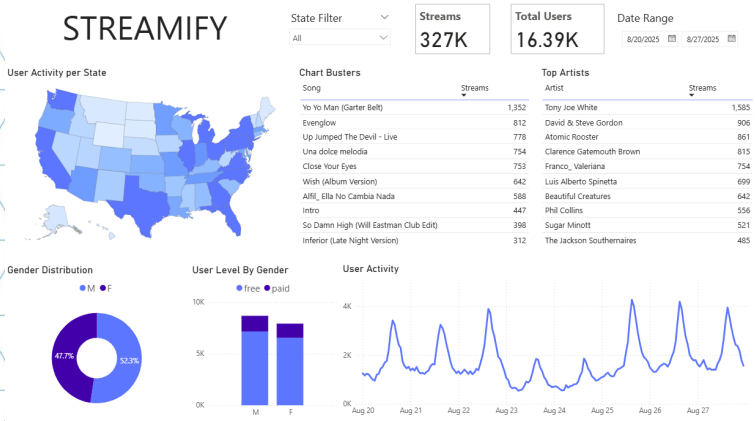
Transformation & Orchestration

- **dbt** — builds star schema inside Snowflake (fact_streams + 5 dim tables + wide_streams view)
- **Airflow** — schedules daily dbt run via DockerOperator



viettel
Theo cách của bạn

Dashboard: Real-time Insights



Key Metrics

- Top artists by play count
- Songs played over time (hourly)
- Geographic distribution (US map)
- Free vs. paid listener split

Connected to `wide_streams` view — pre-joined, no DAX joins needed

viettel
Theo cách của bạn

Agenda

Conclusion

Key Takeaways

1. **Kafka** decouples producers from consumers — swap the processing engine without touching the rest of the pipeline
2. **Flink** delivers sub-10 ms latency, native event-time semantics, and exactly-once guarantees — outperforming Spark for stateful real-time workloads
3. **ELT pattern** keeps concerns separate: Flink loads raw events, dbt transforms inside Snowflake, Airflow orchestrates

Future Directions

- **Flink CDC** — replace Eventsim with Debezium for real database change streams
- **Schema evolution** — Confluent Schema Registry + Avro
- **Real-time ML** — Flink ML model for song recommendations
- **Cloud** — migrate to Kinesis + Confluent Cloud

- THE END -

Thank you for your attention

Contact:
Ngo Quang Hieu — Viettel Digital Talent 2026